Machine Learning

Revision Sheet

Every topic on its own page • formulas, method, worked example, traps

How to use this sheet

- **One topic per page.** Nothing splits across pages — read a page, close the book, and try to reproduce the formula box from memory.
- **Read in this order on each page:** the italic line (what the topic *is*) → the formula box → "how to approach" → the worked example → the traps.
- **The dashed "My notes" box is yours.** Add the variant that caught you out, a formula your coaching notes state differently, or the page number of a question you got wrong.
- **Traps in the orange box are the marks people actually lose.** Re-read only those the night before the exam.

Contents

1. Linear & Multiple Regression
2. Ridge, LASSO & the Bias-Variance Trade-off
3. Logistic Regression & Classification
4. Bayes & Naive Bayes Classifier
5. Discriminant Analysis, k-NN & Decision Trees
6. Support Vector Machines & Kernels
7. Neural Networks & Gradient Descent
8. Model Evaluation & Cross-Validation
9. Clustering: k-means & Hierarchical
10. PCA & Dimensionality Reduction

1. Linear & Multiple Regression

Fit a line by minimising squared error. Everything else in this topic is a consequence of that one choice.

KEY FORMULAS

Simple regression slope and intercept

$$\hat{\beta}_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Multiple regression (normal equations)

$$\hat{\boldsymbol{\beta}} = (X^\mathsf{T} X)^{-1} X^\mathsf{T} y$$

Goodness of fit

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} \quad \text{(simple case: } R^2 = r^2\text{)}$$

WHAT YOU MUST KNOW

- The OLS line **always passes through** $(\bar{x}, \bar{y})$.
- With an intercept, the residuals sum to zero and are orthogonal to every column of X.
- Adding a predictor can **never decrease** training R^2; adjusted R^2 can fall.
- $(X^\mathsf{T} X)^{-1}$ exists only if the columns of X are independent — perfect collinearity breaks OLS.

HOW TO APPROACH A QUESTION

1. Compute $\bar{x}, \bar{y}$ first; the slope formula is built from deviations.
2. For R^2 questions, get RSS and TSS rather than recomputing the fit.
3. For "what happens if we add a variable / rescale x", reason from the properties, not by refitting.

WORKED EXAMPLE

Q. $x = (1, 2, 3, 4, 5)$, $y = (2, 4, 5, 4, 5)$. Find $\hat{\beta}_1$ and $\hat{\beta}_0$.
$\bar{x} = 3$, $\bar{y} = 4$. $\sum(x - \bar{x})(y - \bar{y}) = 6$, $\sum(x - \bar{x})^2 = 10$.

$$\hat{\beta}_1 = \tfrac{6}{10} = \mathbf{0.6}, \qquad \hat{\beta}_0 = 4 - 0.6(3) = \mathbf{2.2}$$

WATCH OUT

- Reporting R^2 as the correlation — it is the correlation *squared*.
- Thinking scaling the features changes the fitted values (it only rescales the coefficients).

MY NOTES

2. Ridge, LASSO & the Bias-Variance Trade-off

Regularization deliberately makes the fit worse on training data so it does better on new data. That trade is the whole topic.

KEY FORMULAS

Ridge objective (L_2)

$$\min_{\beta} \sum_i (y_i - x_i^\top \beta)^2 + \lambda \|\beta\|_2^2$$

Ridge solution

$$\hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top y$$

Error decomposition

$$\mathbb{E}[\text{test error}] = \text{Bias}^2 + \text{Variance} + \sigma^2_{\text{irreducible}}$$

WHAT YOU MUST KNOW

- **Ridge (L_2)** shrinks coefficients toward zero but keeps them all; **LASSO (L_1)** sets some *exactly* to zero, doing feature selection.
- Increasing λ **increases bias and decreases variance**. $\lambda = 0$ is OLS; $\lambda \to \infty$ sends $\beta \to 0$.
- Adding λI makes $X^\top X + \lambda I$ invertible even when $X^\top X$ is singular.
- High variance = overfitting (train error $\ll$ test error); high bias = underfitting (both errors high).

HOW TO APPROACH A QUESTION

1. Identify which norm the penalty uses — that alone answers most MCQs.
2. For "what happens as λ grows", quote bias up / variance down / coefficients shrink.
3. Choose λ by cross-validation, never by training error.

WORKED EXAMPLE

Q. Ridge with weight 0.2 on $\|w\|_2^2$, MAE loss, $w = [-3, 4]^\top$, $x = [1, 2]^\top$, $y_{\text{true}} = x_1 + x_2$. Total loss?
$y_{\text{pred}} = -3 + 8 = 5$, $y_{\text{true}} = 3$, so MAE $= |5 - 3| = 2$. Penalty $= 0.2(9 + 16) = 5$.

$$\text{loss} = 2 + 5 = \mathbf{7.00}$$

WATCH OUT

- Saying ridge uses the L_1 norm — it is L_2.
- Claiming regularization reduces bias; it *increases* bias to cut variance.

MY NOTES

3. Logistic Regression & Classification

A linear score squashed through a sigmoid to give a probability. The boundary is still a straight line.

KEY FORMULAS

Model

$$P(y{=}1 \mid x) = \sigma(w^{\mathsf{T}}x + b), \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

Useful sigmoid facts

$$\sigma(0) = 0.5, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$$

The logit is linear

$$\log \frac{P(y{=}1)}{P(y{=}0)} = w^{\mathsf{T}}x + b$$

Cross-entropy (log) loss

$$L = -\big[y \log \hat{p} + (1-y)\log(1-\hat{p})\big]$$

WHAT YOU MUST KNOW

- The decision boundary $P = 0.5$ is exactly $w^{\mathsf{T}}x + b = 0$ — a **linear** boundary.
- Trained by maximum likelihood; the log loss is **convex**, so there is a single optimum.
- A coefficient w_j multiplies the *odds* by e^{w_j} per unit increase in x_j.
- Logistic regression is **discriminative** (models $P(y \mid x)$); Naive Bayes is generative.

HOW TO APPROACH A QUESTION

1. Compute the score $z = w^{\mathsf{T}}x + b$ first, then apply σ only if a probability is asked for.
2. For classification, you only need the *sign* of z.
3. For loss questions, plug into $-\log \hat{p}$ when $y = 1$.

WORKED EXAMPLE

Q. True label $y = 1$, predicted $\hat{p} = 0.25$. Cross-entropy loss (natural log)?

$$L = -\log(0.25) = \log 4 \approx \mathbf{1.39}$$

WATCH OUT

- Calling the boundary nonlinear because the sigmoid is nonlinear — the *boundary* is linear.
- Training logistic regression with squared error (non-convex and poorly behaved).

MY NOTES

4. Bayes & Naive Bayes Classifier

Turn the classification around with Bayes' rule, then make one bold simplifying assumption to make it computable.

KEY FORMULAS

Bayes' rule

$$P(y \mid x) = \frac{P(x \mid y)\,P(y)}{P(x)}$$

Naive Bayes decision (MAP)

$$\hat{y} = \arg\max_{y} \; P(y) \prod_{j} P(x_j \mid y)$$

WHAT YOU MUST KNOW

- The "naive" assumption is that features are **conditionally independent given the class** — not independent outright.
- It is a **generative** model: it models $P(x \mid y)$ and $P(y)$, unlike logistic regression.
- **Laplace (add-one) smoothing** stops a single unseen word forcing the whole product to zero.
- With equal priors and shared spherical covariance, the boundary is **linear**.

HOW TO APPROACH A QUESTION

1. Write the prior $P(y)$ and each likelihood $P(x_j \mid y)$ separately.
2. Multiply within each class; compare across classes — you rarely need the denominator $P(x)$.
3. Normalise only if the question asks for an actual probability.

WORKED EXAMPLE

Q. $P(\text{spam}) = 0.4$, $P(\text{'win'} \mid \text{spam}) = 0.5$, $P(\text{'win'} \mid \text{ham}) = 0.1$. Find $P(\text{spam} \mid \text{'win'})$.

$$\frac{0.4 \times 0.5}{0.4 \times 0.5 + 0.6 \times 0.1} = \frac{0.20}{0.26} \approx \mathbf{0.77}$$

WATCH OUT

- Forgetting the prior and comparing likelihoods only.
- Reading the naive assumption as "the features are independent" (it is *given the class*).

MY NOTES

5. Discriminant Analysis, k-NN & Decision Trees

Three very different classifiers. Impurity measures for trees are the most computable part and appear most often.

KEY FORMULAS

Entropy and Gini

$$H = -\sum_i p_i \log_2 p_i, \qquad G = 1 - \sum_i p_i^2$$

Information gain

$$IG = H(\text{parent}) - \sum_c \frac{n_c}{n} H(\text{child}_c)$$

Distances for k-NN

$$\|x - z\|_2 = \sqrt{\sum (x_i - z_i)^2}, \qquad \|x - z\|_1 = \sum |x_i - z_i|$$

WHAT YOU MUST KNOW

- Reference values: 50/50 node $\Rightarrow H = 1$, $G = 0.5$; a pure node $\Rightarrow H = G = 0$.
- **LDA** assumes a *shared* covariance $\Rightarrow$ linear boundary; **QDA** allows different ones $\Rightarrow$ quadratic.
- **k-NN** is lazy (no training), needs feature scaling; $k = 1$ gives zero training error and high variance; larger k smooths.
- A fully grown tree overfits; pruning reduces variance. Trees need no feature scaling.

HOW TO APPROACH A QUESTION

1. For a split, compute the parent impurity, then the weighted child impurities, then subtract.
2. For k-NN, compute all distances, sort, take the k nearest and vote.
3. For LDA vs QDA, decide from the covariance assumption in the stem.

WORKED EXAMPLE

Q. A node with 8 positive and 2 negative examples. Find its Gini and entropy.

$$G = 1 - (0.8^2 + 0.2^2) = 1 - 0.68 = \mathbf{0.32}$$

$$H = -0.8 \log_2 0.8 - 0.2 \log_2 0.2 \approx \mathbf{0.72} \text{ bits}$$

WATCH OUT

- Using $\log_e$ when the question asks for entropy in *bits* ($\log_2$).
- Forgetting to weight child impurities by their sample fractions.

MY NOTES

6. Support Vector Machines & Kernels

Pick the boundary with the widest possible gap. Only the points touching the gap matter.

KEY FORMULAS

Margin width

$$\text{margin} = \frac{2}{\|w\|}$$

Hard-margin primal

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^\mathsf{T} x_i + b) \geq 1$$

Hinge loss

$$\max\big(0,\ 1 - y_i(w^\mathsf{T} x_i + b)\big)$$

Kernels

$$K_{\text{lin}} = x^\mathsf{T} z, \quad K_{\text{poly}} = (x^\mathsf{T} z + c)^d, \quad K_{\text{rbf}} = e^{-\gamma\|x-z\|^2}$$

WHAT YOU MUST KNOW

- Minimising $\|w\|^2$ *maximises* the margin — they are the same objective.
- Removing a point that is **not** a support vector does not change the boundary at all.
- Large C penalises misclassification heavily $\Rightarrow$ narrow margin, overfitting risk.
- The kernel trick computes inner products in a high-dimensional space *without* ever mapping the data.

HOW TO APPROACH A QUESTION

1. Identify whether the question is about the margin, the support vectors, or C/γ.
2. For kernel values, just substitute into the kernel formula — these are one-line NAT questions.
3. Remember $K_{\text{rbf}}(x, x) = 1$ always.

WORKED EXAMPLE

Q. $x = (1, 2)$, $z = (2, 0)$, $c = 1$, $d = 2$. Find the polynomial kernel value.

$$x^\mathsf{T} z = 1(2) + 2(0) = 2; \qquad (2 + 1)^2 = \mathbf{9}$$

WATCH OUT

- Thinking all training points influence the boundary — only support vectors do.
- Treating raw Euclidean distance $\|x - z\|$ as a valid kernel (it is not).

MY NOTES

7. Neural Networks & Gradient Descent

Counting parameters and taking one gradient step are the two things GATE DA actually asks.

KEY FORMULAS

Parameter count

$$\sum_{\text{layers}} (\text{in} \times \text{out}) \;+\; \text{one bias per neuron}$$

Gradient-descent step

$$w \leftarrow w - \eta \nabla_w L$$

Activations

$$\text{ReLU}(z) = \max(0, z), \qquad \sigma' = \sigma(1-\sigma), \qquad \tanh' = 1 - \tanh^2$$

WHAT YOU MUST KNOW

- A network of **purely linear** layers collapses to a single linear model — nonlinearity is what buys depth.
- A single perceptron cannot represent **XOR**; a hidden layer can.
- Backpropagation is just the chain rule; softmax + cross-entropy has gradient $\hat{p} - y$.
- Too large a learning rate diverges; too small crawls. On $f = \frac{1}{2}w^2$, GD converges for $0 < \eta < 2$.

HOW TO APPROACH A QUESTION

1. For parameter counts, go layer by layer: multiply the two adjacent layer sizes, then add biases *if* the question includes them.
2. For one GD step, compute the derivative at the current point, multiply by η, subtract.
3. Read carefully whether biases are used — it changes the answer.

WORKED EXAMPLE

Q. MLP with layers $30 \to 4 \to 3 \to 1$, *no biases*. Number of learnable parameters?

$$30(4) + 4(3) + 3(1) = 120 + 12 + 3 = \mathbf{135}$$

WATCH OUT

- Adding biases when the question says there are none (or omitting them when there are).
- Counting neurons instead of weights.

MY NOTES

8. Model Evaluation & Cross-Validation

Four numbers from the confusion matrix generate every metric. Learn which denominator each one uses.

KEY FORMULAS

From the confusion matrix

$$\text{Acc} = \frac{TP + TN}{\text{all}}, \quad \text{Prec} = \frac{TP}{TP + FP}, \quad \text{Rec} = \frac{TP}{TP + FN}$$

F1 (harmonic mean)

$$F_1 = \frac{2\,\text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$$

Rates

$$\text{FPR} = \frac{FP}{FP + TN}, \qquad \text{Specificity} = \frac{TN}{TN + FP}$$

WHAT YOU MUST KNOW

- **Precision** divides by everything you *predicted* positive; **recall** by everything that *is* positive.
- On imbalanced data, accuracy is misleading — a 99% -negative dataset gets 99% accuracy by predicting "negative" always.
- ROC plots TPR against FPR; AUC $= 0.5$ is random, 1.0 is perfect.
- k-fold CV trains k times, and every example is used for validation exactly once. LOOCV is $k = n$.

HOW TO APPROACH A QUESTION

1. Write the confusion matrix out with all four counts before computing anything.
2. Pick the metric's denominator deliberately — that is the only place people slip.
3. Never tune hyperparameters on the test set; use CV on the training data.

WORKED EXAMPLE

Q. $TP = 40$, $FP = 10$, $FN = 20$, $TN = 30$. Give accuracy, precision and recall.

$$\text{Acc} = \tfrac{70}{100} = \mathbf{0.70}, \quad \text{Prec} = \tfrac{40}{50} = \mathbf{0.80}, \quad \text{Rec} = \tfrac{40}{60} \approx \mathbf{0.67}$$

WATCH OUT

- Swapping the precision and recall denominators.
- Quoting accuracy on a heavily imbalanced problem as evidence of a good model.

MY NOTES

9. Clustering: k-means & Hierarchical

Unsupervised grouping. k-means needs k in advance and finds only a local optimum; hierarchical builds a whole tree.

KEY FORMULAS

k-means objective

$$\min \sum_{c} \sum_{x \in C_c} \|x - \mu_c\|^2 \quad \text{(within-cluster SSD)}$$

Lloyd's algorithm

assign to nearest centroid $\rightarrow$ recompute centroids as means $\rightarrow$ repeat

WHAT YOU MUST KNOW

- k-means converges to a **local** optimum only — results depend on initialisation (use k-means++ or restarts).
- It assumes roughly **spherical, similar-sized** clusters, and is sensitive to feature scaling.
- **Single linkage** uses the *minimum* pairwise distance (chains, elongated clusters); **complete linkage** the *maximum* (compact clusters).
- Its clusters are convex (Voronoi cells): the midpoint of two same-cluster points is in that cluster.

HOW TO APPROACH A QUESTION

1. For hierarchical questions, compute all pairwise distances and merge the closest pair first.
2. Check which distance metric is specified — Manhattan and Euclidean give different merges.
3. For k-means by hand, alternate assign/update and stop when assignments stop changing.

WORKED EXAMPLE

Q. $P_1(2,3,-1)$, $P_2(3,1,1)$, $P_3(5,-2,3)$, $P_4(3,3,3)$. Manhattan single linkage: which pair merges first?
$d(P_1,P_2) = 1{+}2{+}2 = 5$; $d(P_2,P_3) = 2{+}3{+}2 = 7$; $d(P_3,P_4) = 2{+}5{+}0 = 7$; $d(P_2,P_4) = 0{+}2{+}2 = 4$.

smallest is 4 $\Rightarrow$ $\mathbf{P_2}$, $\mathbf{P_4}$

WATCH OUT

- Using Euclidean distance when the question specifies Manhattan.
- Claiming k-means finds the global optimum.

MY NOTES

10. PCA & Dimensionality Reduction

Rotate the data onto the directions of greatest variance, then keep only the first few. Unsupervised.

KEY FORMULAS

Principal components

eigenvectors of the covariance matrix, ordered by λ

Variance explained

$$\frac{\lambda_i}{\sum_j \lambda_j} \qquad \text{(cumulative for the first } k\text{)}$$

Link to SVD

$$A = U\Sigma V^{\mathsf{T}}, \qquad \lambda_i = \sigma_i^2/n$$

WHAT YOU MUST KNOW

- Principal components are **mutually orthogonal** — the angle between any two is 90°.
- **Centre the data first** (and usually scale); PCA on uncentred data is meaningless.
- Keep the *largest*-eigenvalue components; the last components carry the least information.
- PCA is **unsupervised** (ignores labels); **LDA** is the supervised analogue.

HOW TO APPROACH A QUESTION

1. Sum the eigenvalues to get total variance, then divide to get proportions.
2. For "how many components for $X\%$ variance", take a running cumulative sum.
3. For geometry questions, remember orthogonality answers most of them instantly.

WORKED EXAMPLE

Q. Covariance eigenvalues $7, 2, 1$. How many components retain at least 85% of the variance?

$$\tfrac{7}{10} = 70\%; \qquad \tfrac{7+2}{10} = 90\% \ \geq 85\%$$

So **2** components suffice.

WATCH OUT

- Keeping the smallest-eigenvalue components (they carry the least information).
- Forgetting to centre the data before computing the covariance.

MY NOTES